



VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Chapter 4: REPETITION STRUCTURES

September 2025

Content

- | | |
|---|--|
| 1 | Introduction to repetition structures |
| 2 | <i>while</i> loops |
| 3 | <i>for</i> loops |
| 4 | <i>Nested loops</i> |
| 5 | <i>do-while</i> loops |
| 6 | <i>break, continue and null</i> statements |

Content

1 Introduction to repetition structures

2 *while* loops

3 *for* loops

4 *Nested loops*

5 *do-while* loops

6 *break, continue and null* statements

Repetition structures

- C++ provides three different forms of repetition structures:
 - *while* structure
 - *for* structure
 - *do-while* structure
- Each of these structures requires a condition that must be evaluated.
- The condition can be tested at either
 - (1) the beginning or
 - (2) the end of the repeating section of code.
- If the test is at the beginning of the loop, the type of loop is a *pre-test loop*. If the test is at the end of the loop, the type of loop is a *post-test loop*.

Fixed count loop and variable condition loop

- In addition to where the condition is tested, *repeating* sections of code are also classified.
- In a *fixed count loop*, the condition is used to keep track of how many repetitions have occurred. In this kind of loops, a fixed number of repetitions are performed, at which point the repeating section of code is exited.
- In many situations, the exact number of repetitions are not known in advance. In such cases, a *variable condition loop* is used.
- In a *variable condition loop*, the tested condition does not depend on a count being achieved, but rather on a variable that can change interactively with each pass through the loop. When a specified value is encountered, regardless of how many iterations have occurred, repetitions stop.

Finite loop and Infinite loop

- A **finite loop** is a sequence of instructions that repeats a limited number of times. It stops when a specific condition is met or after a set number of iterations.
- An **infinite loop** is a sequence of instructions that **continues indefinitely** unless an external action occurs e.g., turning off the power or stopping the program (Wikipedia).
- Types of infinite loops:
 - **Intentional infinite loops**
 - Deliberately created for **continuous processing**.
 - Used in systems like servers, monitoring applications, or embedded systems.
 - **Unintentional infinite loops**
 - Caused by **programming errors**.
 - Usually undesirable and can lead to: High CPU usage, memory issues, unresponsive programs.

Content

1 Introduction to repetition structures

2 ***while* loops**

3 *for* loops

4 *Nested loops*

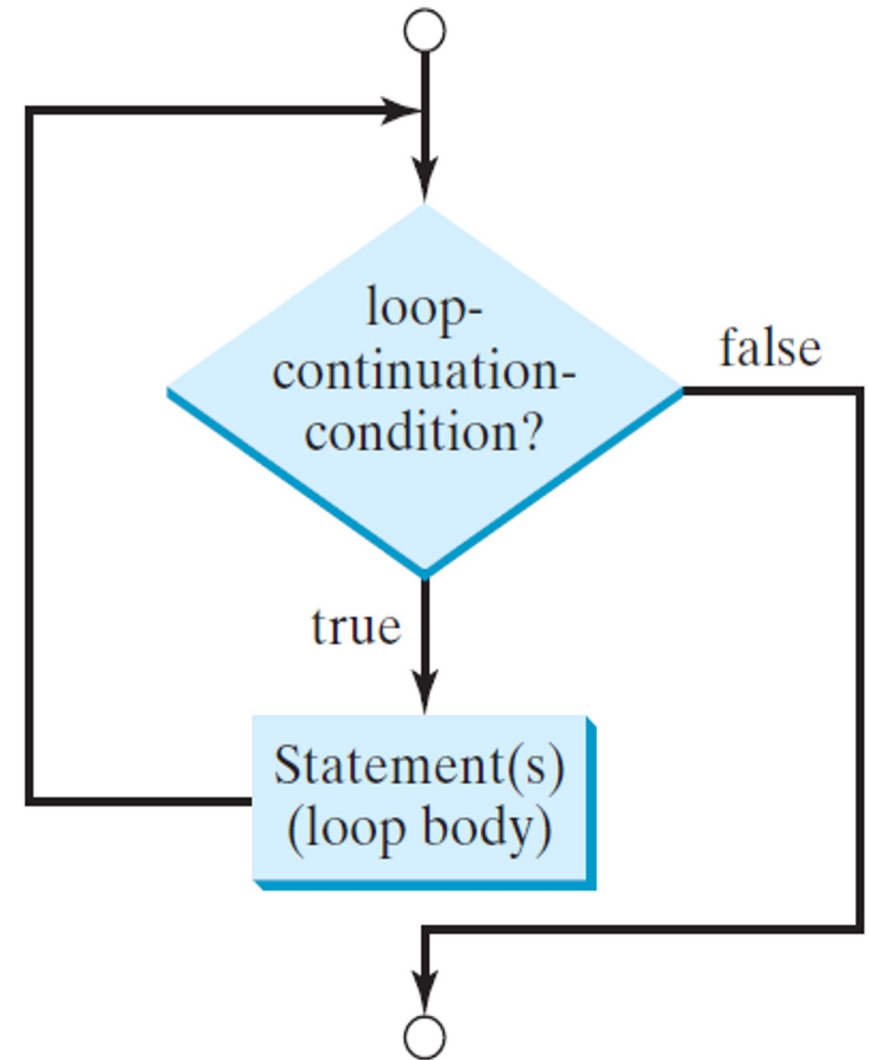
5 *do-while* loops

6 *break, continue and null* statements

while loops

- The *while* statement is used for repeating a statement or series of statements as long as a *given conditional expression* is evaluated to **true**.
- The syntax for the *while* statement:

```
while (<condition-expression>) {  
    <statements>;
```



Example 4.2.1

```
// This program prints out the numbers from 1 to 10
#include <iostream>
using namespace std;
int main() {
    int count;
    count = 1;                // initialize count
    while (count <= 10) {
        cout << count << " ";
        count++;              // increment count
    }
    return 0;
}
```

- The output of the above program:

1 2 3 4 5 6 7 8 9 10

Example 4.2.1

- In the above program, the *loop* incurs a *counter-controlled repetition*.
- Counter-controlled repetition requires:
 - the name of a **control** variable (the variable count)
 - the **initial value** of the control variable (count is initialized to 1 in this case)
 - the **condition** that tests for the final value of the control variable (i.e., whether looping should continue) ;
 - the **increment** (or **decrement**) by which the control variable is modified each time through the loop.

Sentinels

- In programming, data values used to indicate either the start or end of a data series are called *sentinels*.
- The *sentinels* must be selected so as not to conflict with legitimate data values.

Example 4.2.2

```
#include <iostream>
using namespace std;
// sentinel value
const int HIGHGRADE = 100;
int main()
{
    float grade, total;
    grade = 0;
    total = 0;
    cout << "\nTo stop entering grades,"
        << " type in any number
        << " greater than 100.\n\n";

    cout << "Enter a grade: ";
    cin >> grade;
    while (grade <= HIGHGRADE){
        total = total + grade;
        cout << "Enter a grade: ";
        cin >> grade;
    }
    cout << "\nTotal of the grades is "
        << total << endl;
    return 0;
}
```

- In the above program, invalid grades are simply ignored and only valid grades are added to the total.

Content

1 Introduction to repetition structures

2 *while* loops

3 *for* loops

4 *Nested loops*

5 *do-while* loops

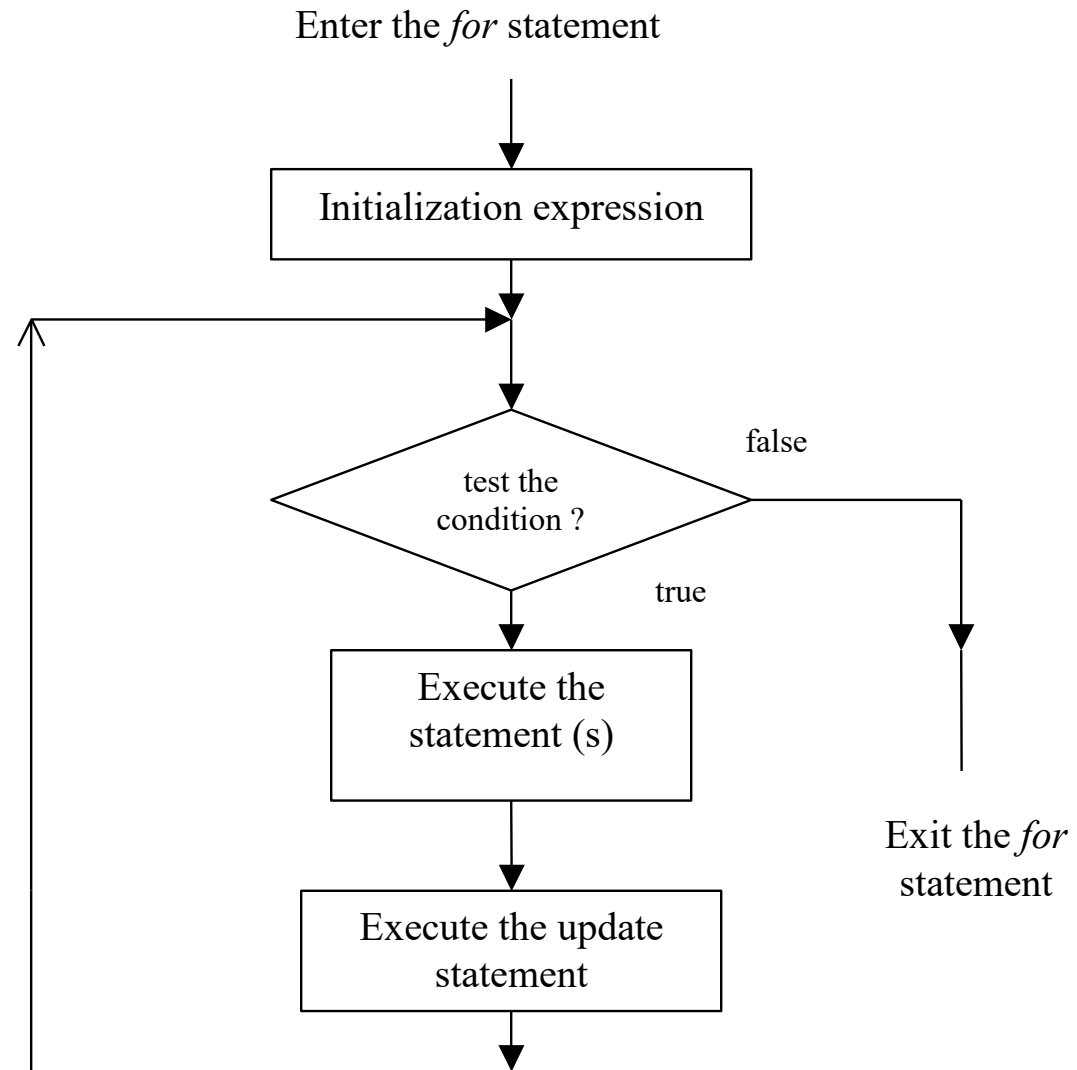
6 *break, continue and null* statements

for loops

- The *for* statement is used for repeating a statement or series of statements as long as a *given conditional expression evaluates to true*.
- One of the main differences between *while* statement and for statement is that in addition to a condition, you can also include code in the for statement:
 - to initialize a counter variable and
 - changes its value with each iteration
- The syntax of the for statement:

```
for (<initialization-expression>; <condition>; <update statement>)  
{  
    <statement(s)>;  
}
```

for loops



Example 4.3.1

```
// This program prints the even number from 2 to 20
#include <iostream>
using namespace std;
int main() {
    int count;
    for (count = 2; count <= 20; count = count + 2)
        cout << count << " ";
    return 0;
}
```

- The output of the above program:

2 4 6 8 12 14 16 18 20

Example 4.3.2

- In this example, we solve this problem:
 - A person invests \$1000.00 in a saving account with 5 percent interest.
 - Assuming that all interest is left on deposit in the account, calculate and print the amount of money in the account at the end of each year for 10 years.
 - Use the following formula for determining these amounts: $a = p(1 + r)^n$
 - where:
 - p is the original amount invested,
 - r is the annual interest rate
 - n is the number of years
 - a is the amount on deposit at the end of the n th year.

Example 4.3.2

```
#include <iostream>
#include <iomanip>
#include <cmath>
using namespace std;
int main() {
    double amount = 0, principal = 1000.0, rate = 0.05;
    cout << "Year" << setw(21) << "Amount on deposit" << endl;
    cout << setiosflags(ios::fixed | ios::showpoint) << setprecision(2);
    for (int year = 1; year <= 10; year++) {
        amount = principal * pow(1.0 + rate, year);
        cout << setw(4) << year << setw(21) << amount << endl;
    }
    return 0;
}
```



Example 4.3.2

- The output of the program:

Year	Amount on deposit
1	1050.00
2	1102.50
3	1157.62
4	1215.51
5	1276.28
6	1340.10
7	1407.10
8	1477.46
9	1551.33
10	1628.89

Content

1 Introduction to repetition structures

2 *while* loops

3 *for* loops

4 *Nested loops*

5 *do-while* loops

6 *break, continue and null* statements

Nested loops

- In many situations, it is convenient to use a loop *contained* within another loop. Such loops are called *nested loops*.
- The output of the program:

```
i is now 1
  j = 1  j = 2  j = 3  j = 4
i is now 2
  j = 1  j = 2  j = 3  j = 4
i is now 3
  j = 1  j = 2  j = 3  j = 4
i is now 4
  j = 1  j = 2  j = 3  j = 4
i is now 5
  j = 1  j = 2  j = 3  j = 4
```

- **Example 4.4.1:**

```
#include <iostream>
using namespace std;
int main() {
    const int MAXI = 5;
    const int MAXJ = 4;
    int i, j;
    // start of outer loop
    for(i = 1; i <= MAXI; i++) {
        cout << "\n i is now " << i
             << endl;
        // start of inner loop
        for(j = 1; j <= MAXJ; j++)
            cout << "  j = " << j;
        // end of inner loop
    } // end of outer loop
    cout << endl;
    return 0;
}
```

Content

1 Introduction to repetition structures

2 *while* loops

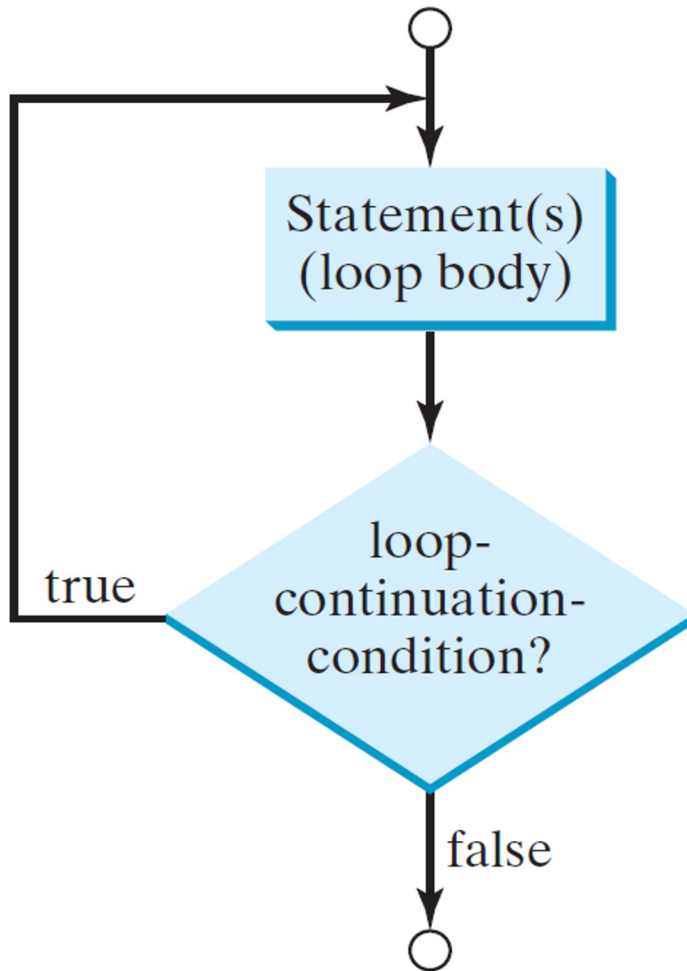
3 *for* loops

4 *Nested loops*

5 *do-while* loops

6 *break, continue and null* statements

do-while loops



- *do-while* statement is used to create post-test loops.
- The syntax:

```
do {  
    // Loop body;  
    <Statement(s)>;  
}  
while (<loop-continuation-condition>);
```

Example 4.5.1

```
#include <iostream>
using namespace std;
int main() {
    int max, sum = 0, digit;
    digit = 2;
    cout << "Enter a number: \n";
    cin >> max;
    do {
        sum = sum + digit;
        digit = digit + 2;
    }
    while (digit <= max);
    cout << "2 + 4 + ... + " << max
         << " sum = " << sum
         << endl;
    return 0;
}
```

- A program to find the sum of even numbers: $2 + 4 + \dots + n$.

- Output of the program:

Enter a number:

10

2 + 4 + ... + 10 sum = 30

Content

-
- | | |
|---|---|
| 1 | Introduction to repetition structures |
| 2 | <i>while</i> loops |
| 3 | <i>for</i> loops |
| 4 | <i>Nested loops</i> |
| 5 | <i>do-while</i> loops |
| 6 | <i>break, continue and null statements</i> |

break statement

- The ***break*** statement causes an exit from the innermost enclosing loop.
- Example 5.6.1:

```
while (count <= 10) {  
    cout << "Enter a number: ";  
    cin >> num;  
    if (num > 76) {  
        cout << "you lose!\n";  
        break;  
    }  
    else  
        cout << "Keep on trucking!\n";  
    count++;  
}  
//break jumps to here
```

***continue* statements**

- The *continue* statement halts a looping statement and restarts the loop with a new iteration.

- Example 5.6.2:

```
int count = 0;
while (count < 30) {
    cout << "Enter a grade: ";
    cin >> grade;
    if (grade < 0 || grade > 100)
        continue;
    total = total + grade;
    count++;
}
```

- In the above program, invalid grades are simply ignored and only valid grades are added to the total.

The *null* statement

- All statements must be **terminated** by a **semicolon**. A semicolon with nothing preceding it is also a valid statement, called the *null* statement.
- Thus, the statement `;` is a null statement.

- Example 5.6.3:

```
if (a > 0)
    b = 7;
else ;
```

- Example 5.6.4:

```
for (int i = 0; i < 10; i++);
```

Content

- | | |
|---|---|
| 1 | Introduction to repetition structures |
| 2 | <i>while</i> loops |
| 3 | <i>for</i> loops |
| 4 | <i>Nested loops</i> |
| 5 | <i>do-while</i> loops |
| 6 | <i>break, continue</i> and <i>null</i> statements |



VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Thank you!